Kurtosys Studio Product-Page Monorepo — User Guide

Repository: `ksys-studio-pp-template`

One folder is enough. This monorepo includes the nested widget starter (`ksys-studio-widget-template/`), so you duplicate a single root for each client — no sibling template repo. The first decision is **where widgets embed** (Kurtosys DXM vs external CMS). That choice factors builds, deploy paths, and docs without forking the widget source.

Contents

1. Step 0 — Choose host mode (start here)
2. What this repo is
3. Prerequisites
4. Repository layout
5. Initialize a client project
6. DXM vs CMS — how the project is factored
7. Manage apps (list / add / rename)
8. Develop a single app
9. Configure environments
10. DXM mount contract
11. CMS integration specs
12. Build for dev / stg / prd
13. Built-in defaults
14. Security audit reports
15. End-to-end checklist
16. Troubleshooting
17. Regenerate this PDF

Step 0 — Choose host mode (start here)

> **Do this first.** Before adding features or filling GUIDs, decide how the widgets will be embedded. Run init and answer the host question (or pass flags).

```
npm run init
# interactive prompts: host mode → client slug → CMS platform (if needed)

# or non-interactive:
npm run init -- --host dxm --client acme
npm run init -- --host cms --client acme --cms-platform pantheon
npm run init -- --host both --client acme --cms-platform wordpress
```

hostMode	Use when	Bundle behaviour	Primary docs / output
`dxm`	Kurtosys DXM / Studio / Elementor pages	IIFE, **React external**	`hosts/dxm/` · `BuildResult/dxm/…`
`cms`	Client WordPress / Pantheon / custom CMS	IIFE, **React bundled** (self-contained)	`hosts/cms/` · integration specs · `BuildResult/cms/…`
`both`	Same widgets delivered to DXM *and* CMS	Builds both variants from one source	Both host folders · dual `BuildResult` trees

> You do **not** need two monorepos. Host mode only changes packaging, deploy path, and documentation. Widget UI/source stays under `apps/` .

Re-run later with `npm run init -- --host cms --force` (or `npm run host:set`) if the delivery model changes.

1. What this repo is

> **Single-folder template.** Copy `ksys-studio-pp-template` once per client. The widget starter lives at `ksys-studio-widget-template/` inside that copy and is what `npm run apps:add` scaffolds from. You do not need a separate `ksys-studio-widget-template` checkout beside it.

A **generic, reusable monorepo** for embeddable React product-page widgets. Each widget is a Vite app under `apps/` , registered in `studio.config.json` .

Starter apps:

App id	Mount id	Embed global	Dev port
`pageHeader`	`#pageHeader`	`window.pageHeader`	5173
`overview`	`#overview`	`window.overview`	5174
`performance`	`#performance`	`window.performanceApp` *	5175
`portfolioBreakdown`	`#portfolioBreakdown`	`window.portfolioBreakdown`	5176
`literature`	`#literature`	`window.literature`	5177

> * `window.performance` is reserved by the browser; the loader uses `performanceApp` automatically.

2. Prerequisites

- **Node.js** 20+ and **npm**
- **Python 3.10+** for security reports and PDF generation
- Optional: Chrome / Edge for PDF export

> The nested `ksys-studio-widget-template/` ships with this repo. Keep it when you duplicate the folder; `studio.config.json` → `widgetTemplatePath` points at `./ksys-studio-widget-template`.

3. Repository layout

```
ksys-studio-pp-template/
├── USER_GUIDE.html / USER_GUIDE.pdf
├── studio.config.json          ← hostMode + apps registry + widgetTemplatePath
├── ksys-studio-widget-template/ ← in-repo widget starter (used by apps:add)
├── hosts/
│   ├── ACTIVE_HOST_MODE.txt
│   ├── dxm/                    ← DXM checklist & notes
│   └── cms/                    ← CMS embed notes
├── docs/
│   ├── ACCESSIBILITY.md
│   └── integration-specs/      ← one markdown spec per CMS product page
├── scripts/                    ← init, add/rename, build-all
├── tools/                      ← security audit + user-guide PDF
├── BuildResult/
│   ├── dxm/{dev,stg,prd}/
│   └── cms/{dev,stg,prd}/
└── apps/
    └── ksys-<client>-<appId>/
```

4. Initialize a client project

Duplicate **only** `ksys-studio-pp-template`. The nested widget starter comes along automatically — do not copy a second template folder.

1. Copy the whole monorepo and rename it for the client:

   ```
   cp -R ~/Documents/GitHub/ksys-studio-pp-template ~/Documents/GitHub/acme-pp
   cd ~/Documents/GitHub/acme-pp
   # confirm the nested starter is present:
   ls ksys-studio-widget-template/scripts/scaffold-app.sh
   ```

2. **Init host mode** (Step 0).
3. If the client slug is not still `client`, rename folders:

   ```
   npm run client:set -- acme
   ```

   (If you already passed `--client acme` to init, still run `client:set` when folders were created as `ksys-client-*`.)
4. `npm run apps:list` — confirm `hostMode` and app folders.

5. Fill each app's `config/env.config.cjs`.

5. DXM vs CMS — how the project is factored

Shared (always)

- One source tree per widget
- A11y defaults, Tailwind `important`, security pins
- App add / rename / client slug tooling

DXM-only factoring

- `HOST_MODE=dxm` → React **externalized**
- Studio GUIDs + Fundpress / applicationManager URLs
- Outputs under `BuildResult/dxm/<env>/`
- Guidance in `hosts/dxm/README.md`

CMS-only factoring

- `HOST_MODE=cms` → React **bundled** (self-contained for WP/Pantheon)
- Integration spec per product page (Confluence-style)
- Outputs under `BuildResult/cms/<env>/`
- Guidance in `hosts/cms/README.md`

`both`

`npm run build:stg` builds each app twice and collects into both `BuildResult/dxm/…` and `BuildResult/cms/…`.

6. Manage apps

Apps are registered in `studio.config.json` and live under `apps/ksys-<clientSlug>-<appId>/`. Use the commands below from the **monorepo root** (not inside an individual app).

```
npm run apps:list
npm run apps:add -- holdingsExposure 5178
npm run apps:rename -- literature materials
npm run client:set -- acme
```

Add a new app for this project

Use this when the product page needs a widget that is not already in the starter set (for example holdings, distributions, key statistics).

1. Pick a camelCase `appId` (mount id / filename stem) and an optional free Vite port.
2. Scaffold and register it:

```
npm run apps:add -- holdingsExposure 5178
# port is optional — if omitted, the next free port from 5173 is used
```

3. Confirm it appears:

```
npm run apps:list
```

4. Open the new folder and fill config:

```
cd apps/ksys-<clientSlug>-holdingsExposure
# edit config/env.config.cjs  (guid, tokens, containerId, embedGlobalName)
npm install
npm run dev
```

5. Build with the rest of the monorepo when ready:

```
npm run build:stg
```

> `apps:add` copies from the nested `ksys-studio-widget-template/`, writes `apps/ksys-<clientSlug>-<appId>/`, sets the Vite port / starter label, and appends the app to `studio.config.json`. After that, treat it like any other starter app.

> If the host page already has a mount node, keep `containerId` / the DXM `id="…"` in sync with your `appId` (or rename deliberately — see below).

Rename an existing app to match the project

Use this when a starter name does not match the client's language (for example rename `literature` → `materials`, or `portfolioBreakdown` → `holdings`).

1. Rename folder, package references, mount/embed ids, and the registry entry in one step:

```
npm run apps:rename -- literature materials
```

2. Check the result:

```
npm run apps:list
# folder is now apps/ksys-<clientSlug>-materials
# mount / filename / embed global follow the new id (e.g. #materials, materials.js
```

3. Update anything *outside* the repo that still uses the old name:
   - DXM / CMS mount nodes (`id="literature"` → `id="materials"`)
   - Script URLs and App Manager child GUIDs pointing at the old filename
   - CMS integration specs under `docs/integration-specs/`
4. Re-fill or confirm `config/env.config.cjs` still has the correct GUIDs for the renamed app.

> `apps:rename` only renames **one widget id**. To change the client prefix on *every* folder (`ksys-client-*` → `ksys-acme-*`), use `npm run client:set -- acme` instead — that is not an app rename.

Client slug vs app id

Goal	Command	Example result
Brand the whole monorepo for a client	`npm run client:set -- acme`	`ksys-client-overview` → `ksys-acme-overview`
Rename one widget to match the page	`npm run apps:rename -- literature materials`	`…-literature` → `…-materials`
Add a brand-new widget	`npm run apps:add -- holdingsExposure`	new `apps/ksys-acme-holdingsExposure/`

Folder convention: `ksys-<clientSlug>-<appId>/`

7. Develop a single app

```
npm run dev -- overview
```

Local `index.html` simulates App Manager + mount node for DXM-style inputs. For CMS work, the same mount `data-input-*` attributes are still the preferred contract.

8. Configure environments

Each app `config/env.config.cjs`: `dev` / `stg` / `prd` (aliases: `development` → `dev`, `staging` → `stg`, `production` | `prod` → `prd`).

- `guid`, `apiToken`, `savedSearchToken`
- `filename`, `containerId`, `embedGlobalName`

> Do not commit production (prd) secrets to public remotes.

9. DXM mount contract

```
<div
  data-ksys-app-template-id="ksys-app-manager"
  data-input-client-code="…"
  data-input-culture="en-US"
  data-input-product-type="mutualFunds"
></div>
<div id="overview"></div>
<script src="…/overview.js"></script>
```

See `hosts/dxm/README.md`.

10. CMS integration specs

For WordPress / Pantheon (and similar), create **one integration spec per product page** — the same idea as your Confluence “Product Page Strategies / All Regions” templates.

1. Copy `docs/integration-specs/_TEMPLATE.md`
2. Rename to the product page (and region if needed)
3. Fill mount selectors, script URLs, cultures, QA checklists

```
cp docs/integration-specs/_TEMPLATE.md \
   docs/integration-specs/acme-global-equity-us.md
```

See `hosts/cms/README.md` for embed snippets.

11. Build for dev / stg / prd

```
# uses studio.config.json hostMode
npm run build:dev
npm run build:stg
npm run build:prd

# force a single host even if mode is "both"
npm run build:stg:dxm
npm run build:stg:cms
```

Artifacts:

```
BuildResult/dxm/stg/overview/overview.js
BuildResult/cms/stg/overview/overview.js
```

> **Clean artifacts only.** Each app build ends with `clean-dist-artifacts`, which deletes scaffolding (`index.html`, `vite.svg`, manifests, embed-loader, etc.) and keeps only the deployable `<appId>.js` (plus optional `shared.css`). `BuildResult` folders stay equally lean.

> Per-app scripts also exist: `build:dxm:stg`, `build:cms:stg`, etc. Monorepo `build-all` sets `HOST_MODE` and prefers those scripts.

12. Built-in defaults

- **A11y** — WCAG-oriented landmarks, contrast tokens, focus rings, jsx-a11y (see `docs/ACCESSIBILITY.md`)
- **Tailwind** — `@import "tailwindcss" important;`
- **Embed global** — `window.<appId>` (not `YourAppEmbed`)
- **Security pins** — vite, rollup, tar, picomatch, postcss

13. Security audit reports

```
pip install -r tools/requirements-audit-report.txt
npm run audit:security
```

HTML/PDF under `reports/`, scoped to app packages and `BuildResult` bundles.

14. End-to-end checklist

1. Duplicate `ksys-studio-pp-template` → rename for the client (widget starter stays nested)
2. `npm run init -- --host dxm|cms|both --client <slug>`
3. `npm run client:set -- <slug>` if folders still use the old slug
4. Rename starter apps to match the page (`npm run apps:rename -- literature materials`) and/or add missing ones (`npm run apps:add -- holdingsExposure`)
5. Fill `env.config.cjs`
6. If CMS/both: create integration specs from `_TEMPLATE.md`
7. Implement UI; keep a11y patterns
8. `npm run build:stg` → verify `BuildResult/<host>/…`
9. Deploy (Studio upload and/or CMS CDN)
10. `npm run audit:security`

15. Troubleshooting

Symptom	Fix
`hostMode is not set` on build	Run `npm run init` first
CMS page has no React / blank widget	Build with `HOST_MODE=cms` (React must be bundled)
DXM double-React / hook errors	Use `HOST_MODE=dxm` (React externalized)
Styles overridden by theme / Elementor	Confirm Tailwind `important` import in `src/index.css`
`apps:add` cannot find template	Confirm `ksys-studio-widget-template/` was kept when you copied the monorepo, and that `widgetTemplatePath` in `studio.config.json` is `./ksys-studio-widget-template`

16. Regenerate this PDF

```
npm run docs:user-guide
# or: python3 tools/build_user_guide_pdf.py
```

Edit `USER_GUIDE.html` (source of truth), then regenerate the PDF.

Kurtosys Studio product-page monorepo · User guide · Duplicate one folder (widget starter included) · then `npm run init` so host mode factors the rest of the project correctly.